

LeagueAnalyzer - Product Requirements Document (PRD)

Project Overview

LeagueAnalyzer is a League of Legends match analysis platform built using Spring Boot and Riot's Match-V5 APIs.

The long-term vision is to become a coaching platform similar to Replayit.gg, where users can:

- Analyze mistakes automatically
- Generate replay clips
- View replay footage in-browser
- Receive AI-generated coaching feedback
- Visualize recurring gameplay issues

Current development focus is backend analytics and timeline parsing.

Tech Stack

Backend:

- Java
- Spring Boot
- Maven

Data Source:

- Riot Games API
- Match-V5
- Timeline-V5

Future:

- React frontend
 - PostgreSQL
 - Replay processing server
 - GPU clip rendering
 - AI analysis service
-

Current Architecture

backend/ | |—— client/ | |—— RiotApiClient.java | |—— controller/ | |——
SummonerController.java | |—— MatchController.java | |—— analyzer/ | |—— TimeAnalyzer.java |
|—— MapCoordinateConverter.java

Completed Features

1. Riot ID Lookup

Endpoint:

GET

/api/player/{gameName}/{tagLine}

Example:

/api/player/Doublelift/NA1

Returns:

- PUUID
- Game Name
- Tag Line

Uses:

Riot Account-V1 API

2. Match ID Retrieval

Endpoint:

GET

/api/matches/{puuid}

Returns:

Recent match IDs

Example:

```
[ "NA1_5569288322", "NA1_5569268366" ]
```

Uses:

Match-V5

3. Match Details Retrieval

Endpoint:

GET

/api/match/{matchId}

Returns:

Full match metadata

Includes:

- Participants
- Champions
- Teams
- Stats

Uses:

Match-V5

4. Timeline Retrieval

Endpoint:

GET

/api/timeline/{matchId}

Returns:

Timeline JSON

Includes:

- Kills
- Deaths
- Assists
- Objectives
- Item purchases
- Skill levels
- Ward events

Uses:

Timeline-V5

5. Death Analyzer

Endpoint:

GET

/api/deaths/{matchId}/{participantId}

Current Functionality:

Scans timeline for:

type = CHAMPION_KILL

Checks:

victimId == participantId

Returns:

- Death timestamp
- Killer
- Assists
- Raw coordinates
- Converted pixel coordinates

Example Output:

Deaths:

09:56 Killed by Ahri Assists: Nautilus Position: 5226, 8824 Map Pixel: 182, 209

16:26 Killed by Lee Sin Assists: none Position: 2587, 9898 Map Pixel: 92, 173

6. Champion Mapping

Implemented.

Process:

Match Details: participantId -> championName

Timeline: killerId -> championName assistIds -> championNames

Output:

Killed by Ahri

instead of:

Killed by participant 8

7. Coordinate Conversion

Implemented.

Class:

MapCoordinateConverter

Converts Riot map coordinates into image coordinates.

Map Bounds:

MIN_X = -120 MIN_Y = -120 MAX_X = 14870 MAX_Y = 14980

Map Size:

512x512

Output Example:

Position: 5226, 8824

Pixel: 182, 209

Purpose:

Future heatmaps and minimap visualizations.

Known Working Flow

Riot ID ↓ PUUID ↓ Match IDs ↓ Match Details ↓ Timeline ↓ Death Analyzer

All currently functional.

Next Features (Priority Order)

Priority 1 - Map Zone Classification

Goal:

Convert coordinates into meaningful locations.

Examples:

Top Lane Bot Lane Mid Lane River Dragon Pit Baron Pit Blue Jungle Red Jungle

Current status:

Not implemented.

Need:

Coordinate region definitions.

Priority 2 - Objective Context Analyzer

Goal:

Determine objective state at death.

Examples:

"Died 12 seconds before dragon."

"Died while Baron was alive."

"Died before soul point fight."

This becomes first real coaching insight.

Priority 3 - Death Heatmap

Goal:

Plot deaths on Summoner's Rift image.

Input:

Map pixels

Output:

Heatmap image

Use cases:

- Common death locations
 - Overextension detection
-

Priority 4 - Multi-Match Analysis

Goal:

Analyze many matches.

Flow:

Player ↓ Recent matches ↓ Aggregate statistics

Examples:

- Most common death locations
- Average deaths before objectives
- Frequent killers

Priority 5 - Mistake Detection Engine

Goal:

Automatically identify mistakes.

Examples:

Died alone in side lane.

Died before objective.

Died while team was grouped elsewhere.

Overstayed after winning fight.

This becomes LeagueAnalyzer's core value.

Priority 6 - Replay Clip Generation

Goal:

Generate clips around mistakes.

Input:

Timeline event

Example:

Death at 12:42

Output:

Clip from:

12:12 → 12:57

Implementation:

User uploads .rofl replay file.

Server processes replay.

Video clip generated.

Priority 7 - Replay Visualization

Goal:

Web-based replay viewing.

Features:

- Timeline
 - Events
 - Heatmaps
 - Death markers
-

Priority 8 - AI Coach

Goal:

Generate coaching feedback.

Input:

Timeline events Match details Generated clips

Example:

"You died 4 times in river while objectives were spawning. Consider warding deeper and grouping earlier."

Long-Term Vision

LeagueAnalyzer becomes:

Riot API ↓ Timeline Analysis ↓ Mistake Detection ↓ Replay Clip Generation ↓ AI Coaching

Comparable to:

- Replayit.gg
- Mobalytics

- U.GG Insights

but with custom replay generation and AI analysis.

Current Development Status

Phase:

Backend Analytics Foundation

Completion Estimate:

~20-25% of MVP completed

Biggest completed milestone:

Successfully parsing Riot timeline data and generating structured death analysis.